



Q1: what is the output of the parent process?

```
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main() {
5      int fd[2][2]; char buffer[100];
6
7      for (int i = 0; i < 2; i++)
8          pipe(fd[i]);
9
10     for (int i = 0; i < 2; i++) {
11         if (fork() == 0) {
12             close(fd[i][0]);
13             write(fd[i][1], "Hello\n", 7);
14             close(fd[i][1]);
15             return 0;
16         }
17     }
18
19     printf("Received:\n");
20     for (int i = 0; i < 2; i++) {
21         read(fd[i][0], buffer, sizeof(buffer));
22         printf("%s\n", buffer);
23         close(fd[i][0]);
24     }
25     return 0;
26 }
```

Received:

Hello

Hello

Q2: What happens if a child process completes execution, but the parent process keeps executing?

- A) The child process becomes an orphan.
- B) The child process becomes a zombie.**
- C) The child process remains active indefinitely.
- D) The child process is removed from the process table.

Q3: What does the pipe() system call do?

- A) Creates a new process
- B) Generates a pair of file descriptors**
- C) Sends a signal to a process
- D) Allocates memory for process communication

Q4: How many times the print statement will be executed in the following code?

```
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main(void) {
5
6      for (int i = 0; i < 3; i++)
7          fork();
8
9      printf("Hello from process with pid = %d\n", getpid());
10     return 0;
11 }
```

A) 3

B) 4

C) 6

D) 8

Q5: Which of the following code create an orphan process?

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    pid_t child_pid = fork();
    if(child_pid > 0){
        while(1){
            sleep(50);
        }
    }
    else{
        exit(0);
    }
    return 0;
}
```

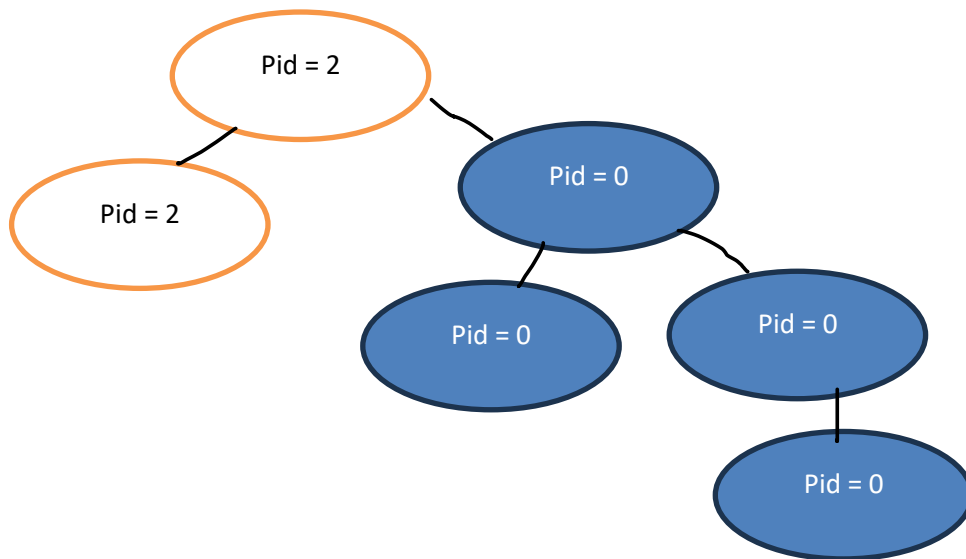
(a)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    int pid = fork();
    if(pid > 0){
        printf("Bye\n");
    }
    else{
        sleep(50);
        printf("Bye\n");
    }
    return 0;
}
```

(b)

Q6: Draw the process tree for the following code

```
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main(void) {
5      int pid = fork();
6      if(pid == 0){
7          fork();
8      }
9      fork();
10     if(pid == 0){
11         printf("Hello from process with pid = %d\n", getpid());
12     }
13
14     sleep(100);
15     return 0;
16 }
```



Q7: How many time the print statement in the above code will be executed?

4 times